1. Simulación de Flujo (XOR)

Esta es la más sencilla para entender cómo un error solo afecta a un carácter.

- **Paso 1:** En **Input**, escribe: ESTE MENSAJE SE MODIFICA SOLO EN LA LETRA QUE QUEREMOS
- **Paso 2:** Busca la operación **XOR** y arrástrala a la "Recipe".
- **Paso 3:** En la configuración de XOR, escribe una clave (ej. clave123).
- **Paso 4:** Arrastra **OTRA** operación **XOR** debajo de la primera, con la **misma clave**.
- **Dinámica:** Ahora cambien una letra en el **Input**. Verán que en el **Output** solo cambia esa letra específica.

2. Simulación de Bloque (AES-ECB)

Aquí verán el "efecto avalancha": un pequeño cambio destruye un bloque de 16 caracteres.

- **Paso 1:** En **Input**, escribe: MENSAJE_BLOQUE_1_MENSAJE_BLOQUE_2_ (exactamente 32 caracteres para que sean dos bloques perfectos).
- **Paso 2:** Busca la operación **AES Encrypt**.
    - **Key:** 1234567890123456 (debe tener 16 caracteres).
    - **Mode:** Cambia CBC por ECB.
- **Paso 3:** Busca la operación **AES Decrypt** y pon la misma configuración (Misma Key y Modo ECB).
- **Dinámica:** cambien la **primera letra** del mensaje.
    - **Resultado:** Los primeros 16 caracteres del Output se volverán "basura" ilegible, pero los siguientes 16 seguirán perfectos.

3. Dinámica Final en CyberChef (Modo "Detective")

Cofiguremos el **AES-ECB** como hicimos antes, pero escribimos este mensaje repetitivo: HOLA_MUNDO_12345HOLA_MUNDO_12345HOLA_MUNDO_12345

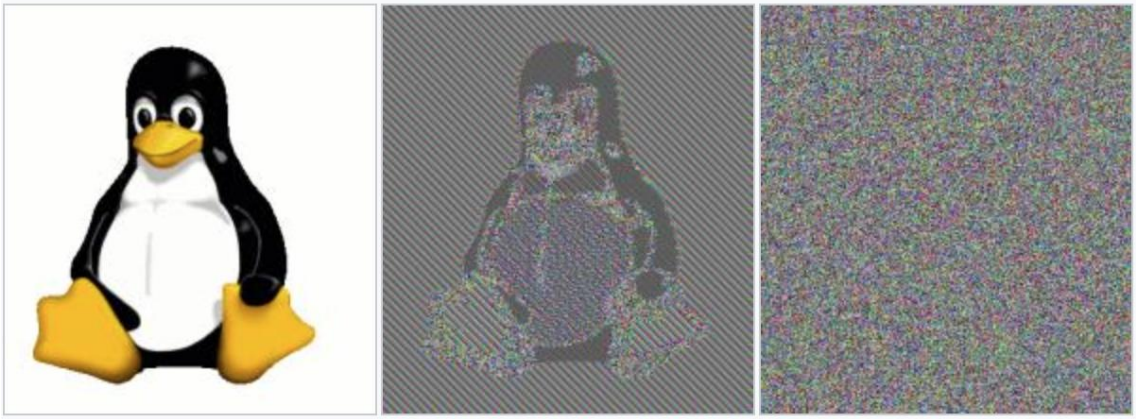
1. **Paso 1:** Que miren el resultado del **AES Encrypt** (antes del Decrypt).
2. **Paso 2:** Observarán que el patrón de símbolos extraños **se repite exactamente cada 16 caracteres**.
3. **Conclusión:** Un atacante no sabe qué dice el mensaje, pero sabe que estás diciendo lo mismo tres veces. ¡Eso es una vulnerabilidad!

4. El Problema del Pingüino (Modo ECB)

Explícales esto: AES-ECB cifra cada bloque de 16 bytes de forma idéntica si el contenido es idéntico.

- **Si tienes un archivo de imagen donde hay un área grande de color blanco (como el fondo de la imagen de Tux), todos esos bloques blancos se cifrarán con el mismo patrón de bits.**

- **Resultado: El mensaje está cifrado (no puedes leer los datos), pero puedes ver la silueta. La estructura del mensaje original se filtra.**



Original image | Encrypted using ECB mode | Modes other than ECB result in pseudo-randomness

Guía de explicación para la imagen:

1. **Imagen Original:** Tux, el pingüino de Linux. Tiene grandes áreas de colores planos (blanco, negro, amarillo).
2. **Modo ECB (El error):** Al cifrarlo, puedes ver que **la silueta de Tux es perfectamente visible**.
    - **¿Por qué sucede?** Como el fondo es todo blanco, el algoritmo divide ese blanco en bloques de 16 bytes. Al ser todos los bloques de entrada iguales, el resultado (ciphertext) de cada bloque es **exactamente el mismo**.
    - **Conclusión:** El modo ECB no oculta patrones. Un atacante sabe dónde hay el mismo contenido sin necesidad de descifrar nada.
3. **Modo Seguro (CBC u otros):** Se ve como "ruido de televisión" total. No hay rastro de la imagen original porque cada bloque depende del anterior, rompiendo cualquier patrón visual.

Resumen:

"Cifrar con **XOR (Flujo)** es como cambiar el color de cada pixel individualmente. Cifrar con **AES-ECB (Bloque)** es como usar un sello: si el papel tiene el mismo dibujo, el sello dejará la misma marca siempre."